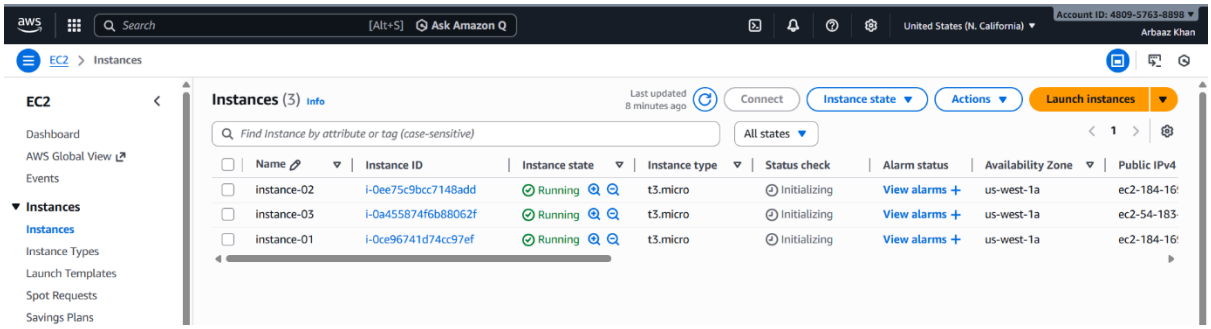
Implement load balancing to distribute traffic across multiple EC2 instances.

Step 1: Launch Three EC2 Instances

Concept:
Multiple EC2 instances act as backend servers to handle requests distributed by the load balancer.

Implementation:

- Launch **3 EC2 instances (Ubuntu)**
- Place them in **public subnets** (or private if NAT is used)
- Attach EC2 security group



Step 3: Configure EC2 Using User Data Script

Concept:
User Data allows automatic configuration of instances during launch, ensuring consistency.

Purpose of Script:

- Install Nginx
- Deploy a sample webpage
- Display instance IP to verify load balancing

User Data Script

```
#!/bin/bash

apt update -y
```

```
apt install nginx -y
PRIVATE_IP=$(hostname -I | awk '{print $1}')
cat <<EOF > /var/www/html/index.html
<!DOCTYPE html>
<html>
<head>
   <title>Load Balancer Test</title>
   <style>
      body {
         font-family: Arial, sans-serif;
         background-color: #f4f6f8;
         text-align: center;
         margin-top: 100px;
      }
      h1 {
         color: #2c3e50;
      }
      p {
         font-size: 20px;
         color: #34495e;
      }
   </style>
</head>
<body>
   <h1>Application Load Balancer Test</h1>
   <p>Served from EC2 Instance</p>
   <p><strong>Private IP:</strong> $PRIVATE_IP</p>
</body>
</html>
```

EOF

systemctl start nginx

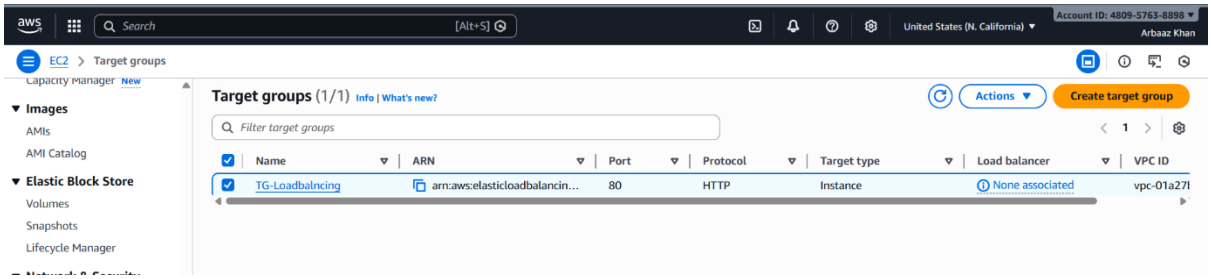
systemctl enable nginx

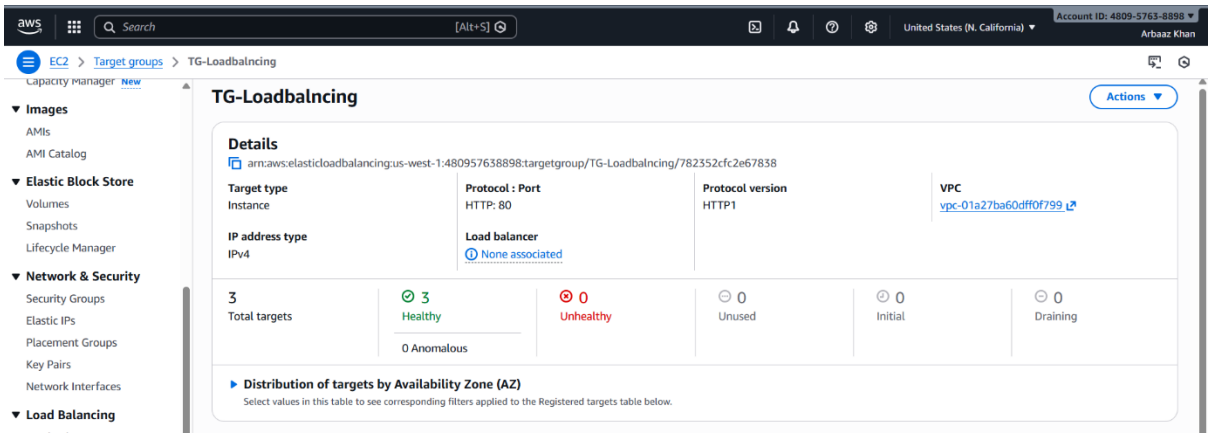
Step 4: Create a Target Group

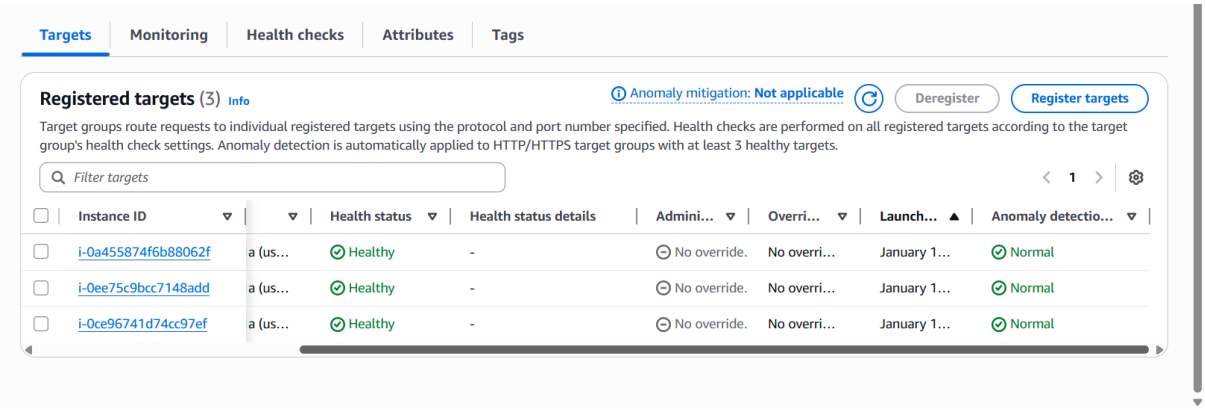
Concept:
A Target Group defines where the load balancer forwards traffic.

Implementation:

- Create a target group for EC2 instances
- Protocol: HTTP
- Port: 80
- Register all three EC2 instances
- Configure health checks







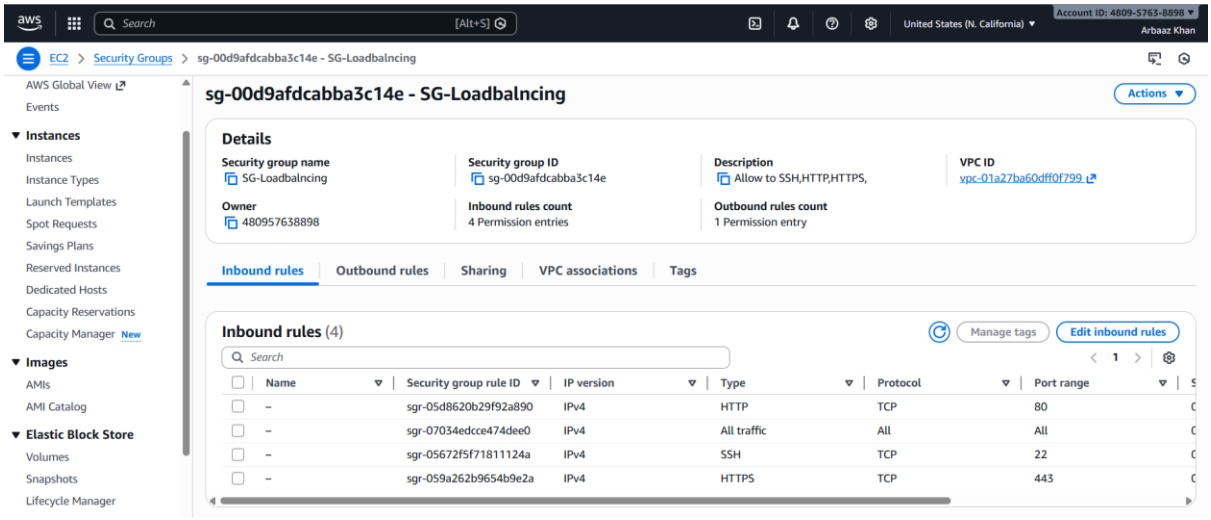
Step 5: Create Security Groups

Concept:
Security groups act as virtual firewalls controlling inbound and outbound traffic.

Implementation:

- **ALB Security Group**
  - Allow inbound HTTP (port 80) from anywhere
- **EC2 Security Group**
  - Allow inbound HTTP (port 80) **only from ALB security group**

This ensures backend instances are protected from direct public access.

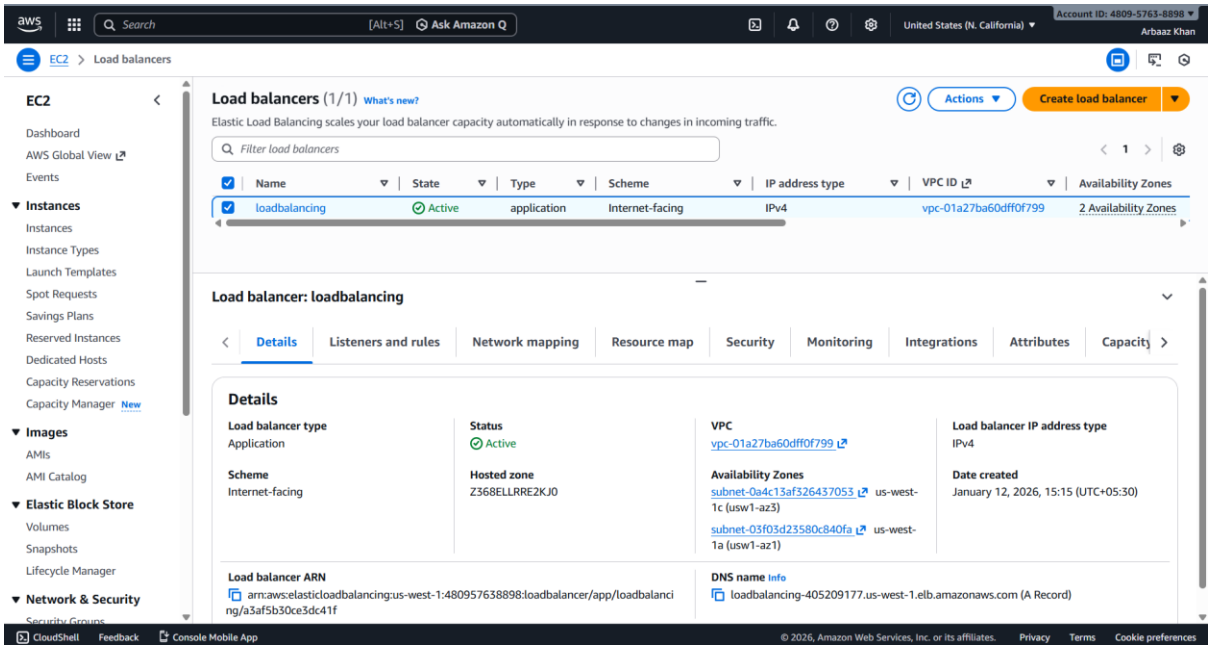


Step 6: Create an Application Load Balancer (ALB)

Concept:
An Application Load Balancer distributes HTTP traffic across multiple targets.

Implementation:

- Create an ALB
- Select public subnets (Multi-AZ)
- Attach ALB security group
- Associate the target group

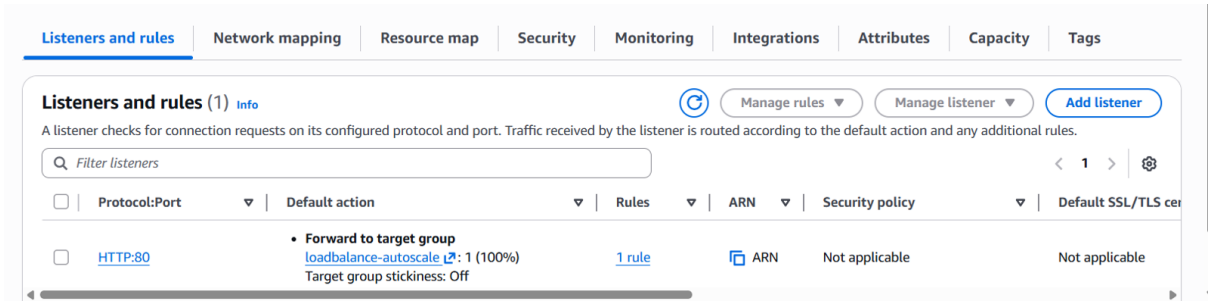


Step 7: Configure Listener Rules

Concept:
Listeners define how the load balancer processes incoming requests.

Implementation:

- Create listener on **HTTP :80**
- Forward traffic to the target group

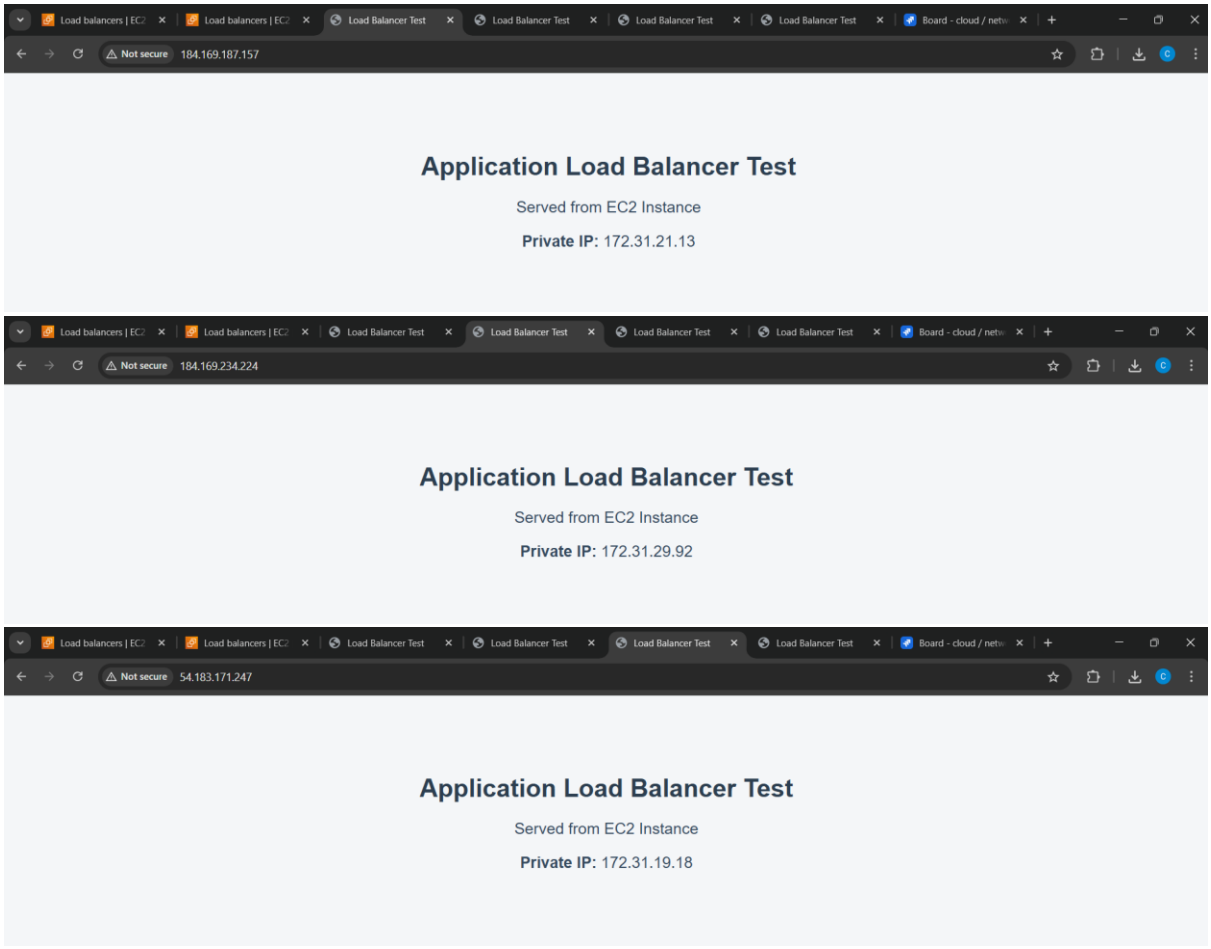


Step 8: Test Load Balancing

Concept:
Testing verifies correct traffic distribution and availability.

Implementation:

- Access ALB DNS name in browser
- Refresh page multiple times
- Observe **IP address change**, confirming load distribution



Step 9: Validate High Availability

Concept:
Load balancing ensures application remains available even if one instance fails.

Validation:

- Stop one EC2 instance
- Refresh ALB URL

- Traffic continues through remaining healthy instances

loadbalancing.mp4

End-to-End Traffic Flow

1. User sends HTTP request
2. Request reaches ALB
3. ALB checks target health
4. Request forwarded to one EC2 instance
5. Nginx serves response
6. Response sent back to user

Outcome

- Traffic evenly distributed across instances
- Improved fault tolerance and scalability
- Backend servers protected by security groups
- Production-ready load balancing architecture achieved